

INTRODUCTION TO C++

WORKSHEET 4



Submitted By: **Sadhabi Dev**

Level : 4

Student ID : 23085140

Cyber Security and Digital Forensics

QUESTION NO.1

1.STL Container Practice: Write a program using STL containers that:

- Uses vector<string> to store names
- Uses map<string, int> to store age against each name

Implements functions to:

- a. Add new name-age pair
- b. Find all people above certain age

Sort and display names alphabetically

SOLUTION:

```
#include <iostream>
#include <vector>
#include <map>
#include <algorithm>
```

```
using namespace std;
```

```
void displayMenu() {
    cout << "\n*****\n";
    cout << "    AGE RECORD MANAGEMENT SYSTEM    \n";
    cout << "*****\n";
    cout << "1. Add name-age pair\n";
    cout << "2. Find people above a certain age\n";
    cout << "3. Display names alphabetically\n";
    cout << "4. Exit\n";
    cout << "=====\n";
}
```

```
void add(map<string, int>& ageMap, vector<string>& nameList) {
    string name;
    int age;
    cout << "\n--- Add Name-Age Pair ---\n";
    cout << "Enter name: ";
    cin >> name;
    cout << "Enter age: ";
    cin >> age;
    nameList.push_back(name);
    ageMap[name] = age;
    cout << "\nEntry added successfully!\n";
}
```

```
void findAboveAge(const map<string, int>& ageMap, int ageLimit) {
    bool found = false;
    cout << "\n--- People above " << ageLimit << " years ---\n";
    for (const auto& person : ageMap) {
        if (person.second > ageLimit) {
            cout << person.first << " (Age: " << person.second << ")\n";
        }
    }
}
```

```

        found = true;
    }
}
if (!found) {
    cout << "No people found above this age limit.\n";
}
}

void showNames(const vector<string>& nameList) {
    cout << "\n--- Sorted Name List ---\n";
    if (nameList.empty()) {
        cout << "No names have been added yet.\n";
        return;
    }

    vector<string> sortedNames = nameList;
    sort(sortedNames.begin(), sortedNames.end());

    for (const auto& name : sortedNames) {
        cout << name << endl;
    }
}

int main() {
    map<string, int> ageMap;
    vector<string> nameList;
    int choice, ageLimit;

    while (true) {
        displayMenu();
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                add(ageMap, nameList);
                break;
            case 2:
                cout << "Enter the age limit: ";
                cin >> ageLimit;
                findAboveAge(ageMap, ageLimit);
                break;
            case 3:
                showNames(nameList);
                break;
            case 4:
                cout << "\nExiting program. Thank you!\n";
                return 0;
            default:
                cout << "Invalid choice! Please try again.\n";

```

```

    }
}
}

```

The output of STL container - Name and Age Management

a. Adding Name Age and Finding people above certain age

```

C:\Users\devsa\OneDrive\Doc x + v

=====
AGE RECORD MANAGEMENT SYSTEM
=====
1. Add name-age pair
2. Find people above a certain age
3. Display names alphabetically
4. Exit
=====
Enter your choice: 1

--- Add Name-Age Pair ---
Enter name: Sadhabi
Enter age: 25

Entry added successfully!

=====
AGE RECORD MANAGEMENT SYSTEM
=====
1. Add name-age pair
2. Find people above a certain age
3. Display names alphabetically
4. Exit
=====
Enter your choice: 1

--- Add Name-Age Pair ---
Enter name: Kusum
Enter age: 19

Entry added successfully!

=====
AGE RECORD MANAGEMENT SYSTEM
=====
1. Add name-age pair
2. Find people above a certain age
3. Display names alphabetically
4. Exit
=====
Enter your choice: 1

--- Add Name-Age Pair ---
Enter name: Amber
Enter age: 30

Entry added successfully!

```

```

C:\Users\devsa\OneDrive\Doc x + v

=====
AGE RECORD MANAGEMENT SYSTEM
=====
1. Add name-age pair
2. Find people above a certain age
3. Display names alphabetically
4. Exit
=====
Enter your choice: 2
Enter the age limit: 16

--- People above 16 years ---
Amber (Age: 30)
Kusum (Age: 19)
Sadhabi (Age: 25)

=====
AGE RECORD MANAGEMENT SYSTEM
=====
1. Add name-age pair
2. Find people above a certain age
3. Display names alphabetically
4. Exit
=====
Enter your choice: 32
Invalid choice! Please try again.

=====
AGE RECORD MANAGEMENT SYSTEM
=====
1. Add name-age pair
2. Find people above a certain age
3. Display names alphabetically
4. Exit
=====
Enter your choice: 2
Enter the age limit: 35

--- People above 35 years ---
No people found above this age limit.

```

b. Displaying names alphabetically

```

C:\Users\devsa\OneDrive\Doc x + v

No people found above this age limit.

=====
AGE RECORD MANAGEMENT SYSTEM
=====
1. Add name-age pair
2. Find people above a certain age
3. Display names alphabetically
4. Exit
=====
Enter your choice: 3

--- Sorted Name List ---
Amber
Kusum
Sadhabi

=====
AGE RECORD MANAGEMENT SYSTEM
=====
1. Add name-age pair
2. Find people above a certain age
3. Display names alphabetically
4. Exit
=====
Enter your choice: 4

Exiting program. Thank you!

Process returned 0 (0x0) execution time : 97.584 s
Press any key to continue.

```

QUESTION NO.2

Stack Problem: Implement a stack using arrays (not STL) that:

- Has basic push and pop operations
- Has a function to find middle element
- Has a function to reverse only bottom half of stack
- Maintain stack size of 10

SOLUTION:

```
#include <iostream>
#include <algorithm>
using namespace std;

class Stack {
private:
    int arr[10];
    int top;

public:
    Stack() : top(-1) {}

    void push(int value) {
        if (top == 9) {
            cout << "Stack Overflow!" << endl;
        } else {
            arr[++top] = value;
        }
    }

    int pop() {
        if (top == -1) {
            cout << "Stack Underflow!" << endl;
            return -1;
        } else {
            return arr[top--];
        }
    }

    int mid() {
        if (top == -1) {
            cout << "Stack is empty!" << endl;
            return -1;
        }
        return arr[top / 2];
    }

    void revBottom() {
        if (top == -1) {
```

```

        return;
    }
    int midIndex = top / 2;
    for (int i = 0; i < midIndex / 2; ++i) {
        swap(arr[i], arr[midIndex - 1 - i]);
    }
}

void print() {
    if (top == -1) {
        cout << "Stack is empty!" << endl;
    } else {
        for (int i = top; i >= 0; --i) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
}

bool empty() {
    return top == -1;
}

bool full() {
    return top == 9;
}

int size() {
    return top + 1;
}
};

int main() {
    Stack stack;

    stack.push(27);
    stack.push(93);
    stack.push(15);
    stack.push(62);
    stack.push(51);
    stack.push(34);
    stack.push(86);
    stack.push(78);
    stack.push(39);
    stack.push(64);

    cout << "\nMiddle element: " << stack.mid() << endl;

    stack.revBottom();
    stack.print();
}

```

```

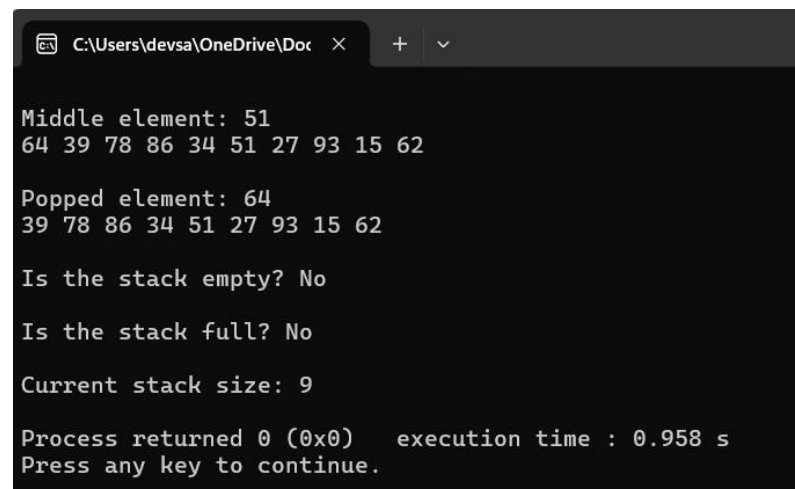
cout << "\nPopped element: " << stack.pop() << endl;
stack.print();

cout << "\nIs the stack empty? " << (stack.empty() ? "Yes" : "No") << endl;
cout << "\nIs the stack full? " << (stack.full() ? "Yes" : "No") << endl;
cout << "\nCurrent stack size: " << stack.size() << endl;

return 0;
}

```

The output of Stack implementation



```

C:\Users\devsa\OneDrive\Doc x + v

Middle element: 51
64 39 78 86 34 51 27 93 15 62

Popped element: 64
39 78 86 34 51 27 93 15 62

Is the stack empty? No
Is the stack full? No

Current stack size: 9

Process returned 0 (0x0)   execution time : 0.958 s
Press any key to continue.

```

QUESTION NO.3

Queue Problem: Implement a queue using arrays (not STL) that:

- Has basic enqueue and dequeue operations
- Has a function to reverse first K elements
- Has a function to interleave first half with second half
- Handle queue overflow/underflow

SOLUTION:

```

#include <iostream>
using namespace std;
class Queue {
public:
    int arr[6];
    int front;
    int back;
    int capacity;

    Queue(int size = 6) : capacity(size), front(0), back(0) {}

    void enqueue(int value) {

```

```

    if (back == capacity) {
        cout << "Queue Overflow!" << endl;
    } else {
        arr[back++] = value;
    }
}

int dequeue() {
    if (front == back) {
        cout << "Queue Underflow!" << endl;
        return -1;
    } else {
        return arr[front++];
    }
}

void reverseFirstK(int K) {
    if (K <= 0 || K > back - front) {
        cout << "Invalid K!" << endl;
        return;
    }

    for (int i = 0; i < K / 2; i++) {
        int temp = arr[front + i];
        arr[front + i] = arr[front + K - i - 1];
        arr[front + K - i - 1] = temp;
    }
}

void interleave() {
    int n = back - front;
    if (n < 2) {
        cout << "Not enough elements to interleave!" << endl;
        return;
    }

    int mid = front + n / 2;
    int temp[6];

    int i = front, j = mid, k = 0;
    while (i < mid && j < back) {
        temp[k++] = arr[i++];
        temp[k++] = arr[j++];
    }

    while (i < mid) {
        temp[k++] = arr[i++];
    }

    while (j < back) {

```

```

        temp[k++] = arr[j++];
    }

    for (int i = 0; i < n; i++) {
        arr[front + i] = temp[i];
    }
}

void printQueue() {
    if (front == back) {
        cout << "Queue is empty!" << endl;
    } else {
        for (int i = front; i < back; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
}

};

int main() {
    Queue q;

    q.enqueue(5);
    q.enqueue(10);
    q.enqueue(15);
    q.enqueue(20);
    q.enqueue(25);

    cout << "\nQueue after enqueue operations: ";
    q.printQueue();

    q.reverseFirstK(3);
    cout << "\nQueue after reversing first 3 elements: ";
    q.printQueue();

    q.interleave();
    cout << "\nQueue after interleaving halves: ";
    q.printQueue();

    cout << "\nDequeued: " << q.dequeue() << endl;
    cout << "\nQueue after dequeue operation: ";
    q.printQueue();

    q.enqueue(30);
    cout << "\nQueue after attempting to enqueue 30: ";
    q.printQueue();

    return 0;
}

```

The output of the Queue implementation

```
C:\Users\devsa\OneDrive\Doc × + v
Queue after enqueue operations: 5 10 15 20 25
Queue after reversing first 3 elements: 15 10 5 20 25
Queue after interleaving halves: 15 5 10 20 25
Dequeued: 15
Queue after dequeue operation: 5 10 20 25
Queue after attempting to enqueue 30: 5 10 20 25 30
Process returned 0 (0x0)   execution time : 4.522 s
Press any key to continue.
```

QUESTION NO.4

Linked List Problem: Create a singly linked list (not STL) that:

- Has functions to insert at start/end/position
- Has a function to detect and remove loops
- Has a function to find nth node from end
- Has a function to reverse list in groups of K nodes

SOLUTION:

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node(int value) : data(value), next(nullptr) {}
};

class LinkedList {
public:
    Node* head;

    LinkedList() : head(nullptr) {}

    void insS(int value) {
        Node* newNode = new Node(value);
        newNode->next = head;
        head = newNode;
    }

    void insE(int value) {
```

```

Node* newNode = new Node(value);
if (!head) {
    head = newNode;
    return;
}
Node* temp = head;
while (temp->next) {
    temp = temp->next;
}
temp->next = newNode;
}

void insP(int value, int pos) {
    if (pos < 1) {
        cout << "\nInvalid position!" << endl;
        return;
    }
    Node* newNode = new Node(value);
    if (pos == 1) {
        newNode->next = head;
        head = newNode;
        return;
    }
    Node* temp = head;
    for (int i = 1; i < pos - 1; ++i) {
        if (!temp) {
            cout << "\nInvalid position!" << endl;
            return;
        }
        temp = temp->next;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}

void findNth(int n) {
    Node* temp = head;
    int length = 0;
    while (temp) {
        length++;
        temp = temp->next;
    }
    if (n > length) {
        cout << "\nInvalid position!" << endl;
        return;
    }
    temp = head;
    for (int i = 1; i < length - n + 1; ++i) {
        temp = temp->next;
    }
}

```

```

    cout << "\nNth from end: " << temp->data << endl;
}

void revK(int k) {
    if (!head || k <= 1) return;
    Node *current = head, *prevFirst = nullptr, *first = nullptr;
    int count = 0;

    while (current) {
        count++;
        current = current->next;
    }

    current = head;
    while (count >= k) {
        first = current;
        prevFirst = nullptr;
        Node* prev = nullptr;
        Node* next = nullptr;
        for (int i = 0; i < k && current; i++) {
            next = current->next;
            current->next = prev;
            prev = current;
            current = next;
        }
        if (!prevFirst) {
            head = prev;
        } else {
            prevFirst->next = prev;
        }
        first->next = current;
        prevFirst = first;
        count -= k;
    }
}

void loopD() {
    if (!head || !head->next) return;
    Node* slow = head;
    Node* fast = head;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {
            cout << "\nLoop detected. Removing loop." << endl;
            slow = head;
            while (slow->next != fast->next) {
                slow = slow->next;
            }
        }
    }
}

```

```

        fast = fast->next;
    }
    fast->next = nullptr;
    return;
}
}
cout << "\nNo loop detected." << endl;
}

void print() {
    if (!head) {
        cout << "\nList is empty!" << endl;
        return;
    }
    Node* temp = head;
    while (temp) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}
};

int main() {
    LinkedList list;

    list.insS(12);
    list.insE(37);
    list.insE(99);
    list.insP(45, 2);
    list.insE(83);

    cout << "\nList after insertions: ";
    list.print();

    list.findNth(3);

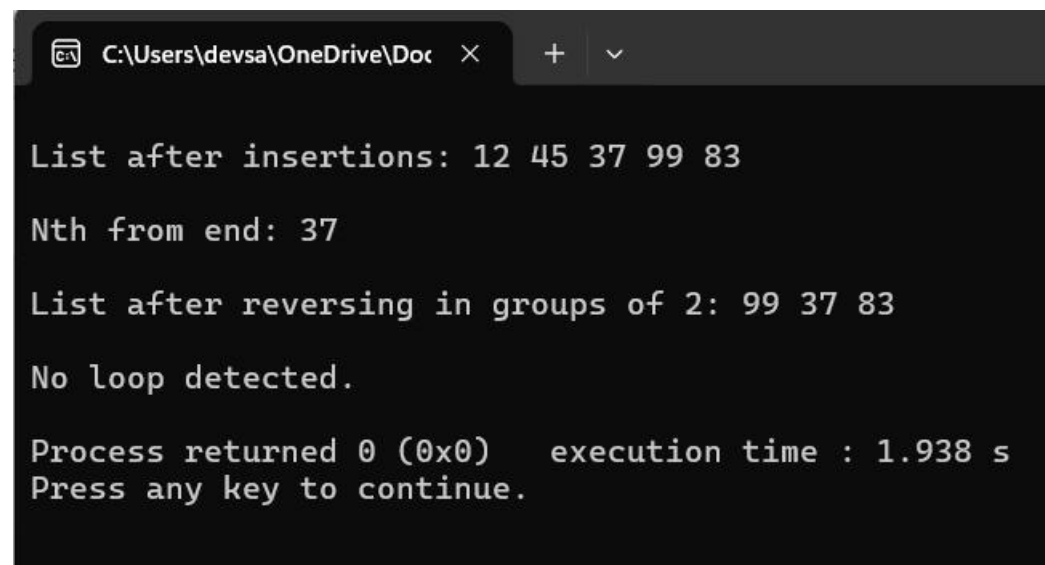
    list.revK(2);
    cout << "\nList after reversing in groups of 2: ";
    list.print();

    list.loopD();

    return 0;
}

```

The output of Linked List implementation



```
C:\Users\devsa\OneDrive\Doc  X  +  v

List after insertions: 12 45 37 99 83

Nth from end: 37

List after reversing in groups of 2: 99 37 83

No loop detected.

Process returned 0 (0x0)    execution time : 1.938 s
Press any key to continue.
```

My GitHub Link: https://github.com/sd6012/cpp_Assignment